Warrant — Appwrite Databases Hackathon plan

Oct 29, 2026 · @dd

Summary

Ship Warrant, a TablesDB app where revoking an AI agent takes effect on its very next write, and submit by **Saturday 23:59 Lagos time**. The hard deadline is Sunday 1 Nov at 07:59 Lagos (31 Oct, 11:59 PM PT), so that leaves 8 hours of buffer.

- **Thesis:** your agent's leash should be a row. Authority lives in a TablesDB row, so the database enforces revocation, not the agent's code.
- **Demo:** two agents run the same task. One click revokes both. The credential agent keeps buying; the warrant agent is rejected by Appwrite on its next write.
- **Hook:** Appwrite closed this same gap for human sessions in October (#14021, #14039). Warrant applies that rule to agents, and avoids an open transaction bug (#14489).
- **Rules that bind:** solo, built from scratch after 19 Oct, one database (TablesDB), public repo plus a 2–3 minute video.

How it scores

Every criterion is carried by one specific thing on screen, not by a claim in the voiceover.

Criterion	What carries it	Proof on screen
Idea and innovation	Revocation as a database guarantee for AI agents	The race: 4 actions after revoke vs 0
Technical execution	Atomic spends, idempotent tool calls, permission-based revoke	The jar: 20 parallel spends, exactly $50.00 spent
Design and experience	Warrant as a paper slip, revoke line, X-ray panel, receipt	One click, both lanes, and the bounce explains itself
Usage of Appwrite Databases	Every guarantee maps to a named TablesDB primitive	Guarantees table in the README and on the receipt

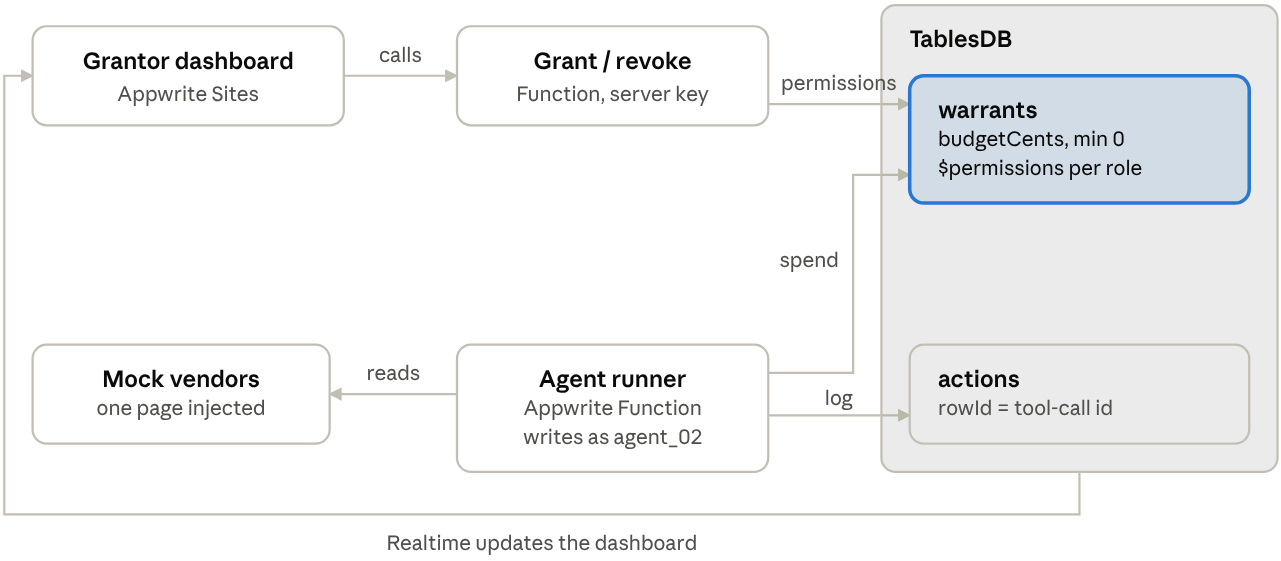
The issue-tracker hook. Two citations go into the video and the README. Neither is a new feature.

- #14039 / #14021: *"The HTTP API refuses that session's next request; realtime kept delivering."* Fixed in October for human sessions. Warrant is the same rule for agents.
- #14489, open: two concurrent transactions can both commit a claim of the same row. Warrant never uses a transaction as a lock.

Architecture

The agent is its own Appwrite user and spends only through its warrant row, so revoking means removing one permission from that row.

Revoking edits one permission on the warrant row



warrant lane · 5 parts, 1 table guarded

The agent can spend only through the warrants row, so the grant/revoke Function editing that row's permissions is the whole leash. The credential lane, not drawn here, skips the row and calls vendors with a key.

Stack: TablesDB (rowSecurity on), Appwrite Functions (Node) for the agent runner and the grant/revoke actions, Realtime for the dashboard, Appwrite Sites for the frontend. One LLM API for the agent's choices.

Tables

Table	Key columns	Permissions
`warrants`	`budgetCents` (int, min 0), `actionsLeft` (int, min 0), `vendors` (string[]), `expiresAt`, `status`, `revokedAt`	grantor read+update; agent read+update until revoked
`actions`	`warrantId`, `lane` (credential / warrant), `tool`, `amountCents`, `outcome`, `error`	rowId = tool-call id; grantor read
`vendors`	`name`, `catalog` (text), `injected` (bool)	read any; one seeded row carries the injection

Guarantees

Guarantee	Primitive
Revoke takes effect on the next write	Row `$permissions`, with the agent writing as its own user
Budget can never go negative	`decrementRowColumn` with `min: 0` (race-free per #13464)
A retried tool call can't charge twice	`rowId` = tool-call id, so a retry gets `row_already_exists`
Nothing committed after revoke, provably	`listRows` on `actions` with `$createdAt > revokedAt` returns 0
Grantor sees everything live	Realtime on `warrants` and `actions`

Traps avoided

- No transaction used as a lock (#14489).
- Never stage transaction operations in parallel with a commit (#14332). The agent's parallel tool calls go straight to `decrementRowColumn`.
- The credential lane is a long-lived **third-party vendor key** checked once at grant time, never an Appwrite JWT. Appwrite already rejects JWTs after session deletion or expiry (#8000).

Screens

Build four screens, in priority order. The mocks are the spec.

1. **The race (hero).** Two lanes, one revoke line, an after-revoke count per lane, and an X-ray panel that shows the rejected call and the permission removed. The video is built around this screen.
2. **Grant.** The warrant as a paper slip (budget meter, vendors, expiry, max actions), with the row and its `$permissions` beside it.
3. **Receipt.** "0 actions committed after revocation", the proof query, the guarantees table and the two issue citations. This is the closing shot.
4. **The jar.** Twenty parallel $10 spends against a $50 warrant: exactly 5 committed, 15 rejected. The naive read-then-write comparison uses a figure you measure, never an invented one.

Scripted scenario mode seeds the same vendors, the same injection and the same timing on every run, so you can record ten takes. The agent stays real; only the stage is fixed.

Schedule

About 55 working hours remain. Each block ends with a test that must pass before the next block starts. All times are Lagos.

Thu 29, 17:00–24:00 — Spike and schema

- [ ] Create the project, database and the three tables with rowSecurity on
- [ ] Create the agent user, and mint its JWT from a Function (`users.createJWT`)
- [ ] As the agent, call `decrementRowColumn(min: 0)` on its warrant: it succeeds
- [ ] Remove `update("user:agent")` via a server key, then repeat the call with the **same** JWT: it must be rejected
- [ ] Fire 20 parallel decrements at $50: exactly 5 succeed
- [ ] Write down the exact error codes Appwrite returns

- Done when: revoke blocks the next write without restarting the agent. If not, switch to Plan B (Risks) tonight.

Fri 30, 09:00–13:00 — Agent runner

- [ ] Agent loop in a Function with three tools: `read_catalog`, `purchase`, `finish`
- [ ] `purchase` writes the `actions` row (rowId = tool-call id), then decrements the warrant

- [ ] Credential lane: same loop, but `purchase` checks a vendor key once and never re-checks
- [ ] Seed vendors, with the injection in Brightline's catalog
- Done when: both lanes complete the task from a script, and a revoke stops only the warrant lane.

Fri 30, 14:00–22:00 — Race screen

- [ ] Grant screen writes the warrant row and starts both lanes
- [ ] Race view with both lanes fed by Realtime
- [ ] Revoke button calls the revoke Function, and the line drops across both lanes
- [ ] After-revoke counters and the X-ray panel for the selected event
- Done when: one click produces 4-vs-0 live, with no page refresh.

Sat 31, 09:00–15:00 — Jar, receipt, scripted mode

- [ ] Jar: fire 20 in parallel; measure the naive read-then-write overspend and record the real number
- [ ] Receipt page runs the proof query live and shows the guarantees table and the issue citations
- [ ] Scripted scenario mode: same seed and timing every run
- Done when: three clean runs in a row.

Sat 31, 15:00–20:00 — Ship

- [ ] Deploy on Appwrite Sites
- [ ] README: thesis, guarantees table, the issue links, how to run, tech stack
- [ ] Record the video (script below), maximum 3 minutes, and upload it to YouTube
- Done when: the live link works in a private window.

Sat 31, 20:00–23:59 — Submit

- [ ] Make the repo public, then submit the repo and video links on the hackathon dashboard
- [ ] Sun 1 Nov, 00:00–07:59: buffer only. Fix only, add nothing.

Scope fence

The idea is locked. Anything not listed under "In" waits until after submission.

In: one agent per lane, one purchasing domain with mock vendors, grant, revoke, race, jar, receipt, scripted mode, README, video.

Out: multiple agents, delegation chains or sub-warrants, an LTL engine, MCP Events, OAuth via Appwrite, real payments, revoke from a phone via QR, any core Appwrite PR before submission.

Cut order if late: cut in this order, item 1 first. The race and the receipt are never cut.

1. X-ray panel polish (keep a plain error line instead)
2. Jar naive-comparison card (keep the jar itself)
3. Grant screen styling (a plain form is fine)
4. The jar
5. Scripted mode (record live and keep the best take)

Risks and fallbacks

The one risk that can sink the demo is whether a permission change stops an agent that is already running, so Thursday's spike tests it first.

Risk	Fallback
Removing `update("user:agent")` does not reject the agent's next write	**Plan B:** the agent joins a team `warrant-0042`, and the row grants `update("team:warrant-0042")`. Revoke deletes the membership. Realtime already re-resolves on membership events (#14021).
The grantor's client can't write a permission for the agent's role	Grant and revoke run in a Function with a server key. The button calls the Function.
`decrementRowColumn` and the `actions` insert can't share one atomic step	Insert the action as `pending` first (idempotent id), decrement, then mark it `committed` or `rejected`. The proof query counts only `committed`.
The model ignores the injected instruction	Scripted mode plays the injected purchase as a compromised agent and labels it that way in the video. Never claim the model was fooled if it wasn't.

Risk	Fallback
The LLM is slow or flaky on camera	Scripted mode, low temperature, and record ten takes.
Time runs out	Follow the cut order. Submit Saturday night regardless.

Video and submission

The video runs 2:30, hard cap 3:00, and ends on proof, not a claim.

Time	On screen	Voiceover
0:00	Grant screen, empty	“This agent can spend my money. What happens when I tell it to stop?”
0:12	The slip fills: $200, three vendors, 30 min	“It doesn’t get a key. It gets a row.”
0:25	Race: both lanes buying normally	Same model, same task, two ways of holding authority.
0:45	The injected vendor page; both agents overspend	“One vendor’s page tells it to buy 40 more hubs.”
1:00	**Revoke.** The line drops. Left: 4 more buys. Right: one bounce	“One click. The left agent never heard it.”
1:20	X-ray: the rejected call and the struck-out permission	“No line of the agent’s code checks revocation. The row does.”
1:45	The jar: 20 in parallel, exactly $50.00	`decrementRowColumn` with `min: 0`
2:05	Receipt: the issue citations	“Appwrite fixed this for human sessions in October. This is the same rule for agents.”
2:20	Receipt: “0 actions” and the query	“Your agent’s leash should be a row.”

Submission checklist

- [ ] Public GitHub repo with a first commit dated 19 Oct or later
- [ ] README with the tech stack and open-source libraries listed (required by the FAQ)

- [ ] YouTube video, 2–3 minutes, set to public or unlisted
- [ ] Live Sites link (optional but worth it)
- [ ] Submitted on the dashboard before Sat 23:59 Lagos

After submission

Turn the build into a contribution and a talk without stepping on anyone's work.

- [ ] Comment on #14489 with your agent-workload reproduction. Don't open a competing PR: the issue was assigned on 29 Oct and fix PR #14574 is open.
- [ ] Publish the pattern as a write-up: "Revocation is a database problem", with the race GIF and the guarantees table
- [ ] Turn the video storyboard into a 10-minute talk: *Your agent's leash should be a row*
- [ ] Link it back to the key-lifecycle paper as the working demo of the maintenance layer